Live on devnet | payer-signed mandate | enforced on-chain | no backend

Faktury, które *płacą się same.*

Self-collecting B2B invoices on Solana. The debtor signs a payment mandate once — the payment pulls itself on the due date, late fees enforce themselves on-chain, and investors can advance cash against unpaid invoices.

HackYeah 2026 · Superteam Poland track program GhEAovtia…fQEhZ1 github.com/zuhdev/termin

Payment terms are *polite fiction.*

69%

of Polish companies pay suppliers late — the 2nd worst rate in the EU (Intrum)

30–90 days

a small subcontractor waits for cash while payroll runs monthly. Chasing payments is manual — and often never happens.

2026

KSeF makes e-invoicing mandatory for every Polish business — invoices go fully digital, but payment terms stay stale.

"The invoice exists on-chain. *Why can't the payment live there too?*"

TARGET USER

Marek — *IT subcontractor*, Kraków, 10 people

TODAY

40% of invoices arrive 30+ days late · ~2h/month spent on reminders and calls

`send invoice → wait → remind → wait → maybe paid`

WITH TERMIN

Send once — the money arrives on the due date, automatically. Late fees accrue on-chain, visibly.

`send once → protocol collects + fees → paid`

The debtor keeps control: they sign the mandate, can revoke before the due date, and fees are transparent and capped by the program.

SOLUTION

Four moves, *all on-chain.*

1

Issuer creates the invoice

Amount, due date, and late-fee terms (e.g. 0.5%/day, capped at 10%) live in a PDA. No signup, no accounts.

2

Payer signs once

One signature approves the program's vault PDA as an SPL delegate: "pull `amount + max fee` on the due date."

3

Collection is permissionless

At due, anyone can trigger it — the issuer, a bot, KSeF middleware. The program signs as delegate and the fee is computed on-chain.

4

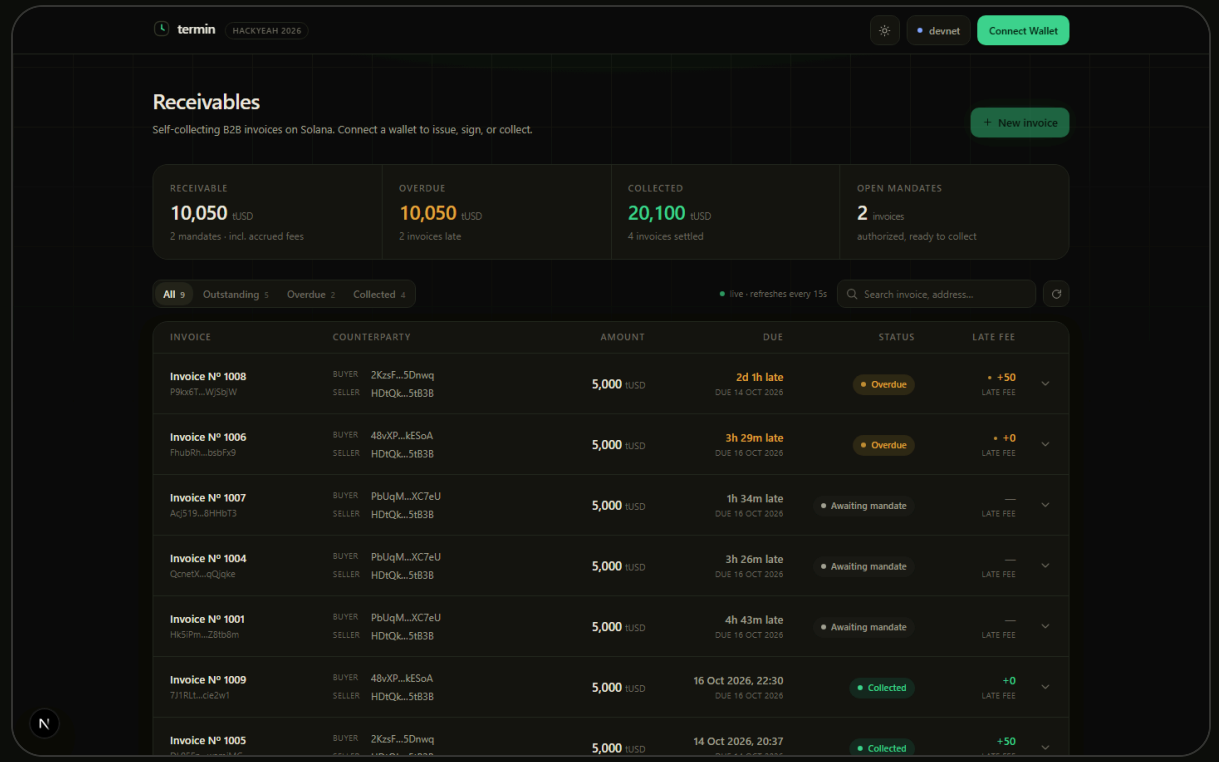
Optional advance (LP)

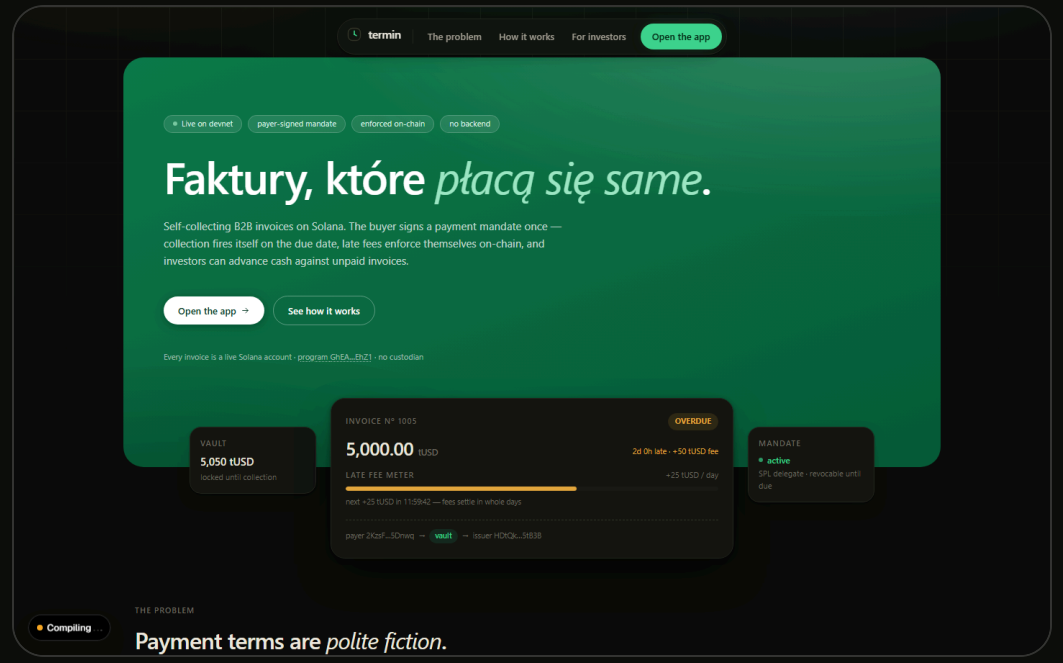
An investor pays the issuer today at a discount; at collection the waterfall is payer → LP → issuer. Positions are priced, not promised.

Like a standing order — *but for invoices, with teeth.*

LIVE ON DEVNET

The console, *wired to the chain.*





Every row is a live Solana account: overdue fee `+50 tUSD` accruing on invoice 1008, open mandates awaiting collection, LP-advanced settlement — no mock data, no database.

3-min walkthrough · [video URL] · explorer.solana.com → program GhEAovtia…fQEhZ1 (devnet)

HOW IT WORKS

7 instructions. *One PDA per invoice.*

issuer —create→ invoice PDA ←authorize— payer (SPL delegate = vault PDA)

due date reached · anyone may collect

payer —amount + fee→ issuer | no LP

payer —payout→ LP and issuer (total − payout) | advanced

late_fee = floor(amount × fee_bps_per_day × full_days_late / 10 000) · capped at fee_cap_bps
enforced by the program — not a promise · transfers via transfer_checked signed by the vault_authority PDA

Cheap enough for *real invoices.*

economics

Fractional-cent fees make a 5,000 tUSD invoice economically viable — unlike L1 smart contracts.

primitive

The SPL token delegate is the native “pull payment” primitive — a mandate is one signature.

permissionless

Collection can be triggered by anyone: the issuer, a cron bot, or KSeF e-invoicing middleware.

disputes

Fee math lives on-chain — no “the reminder never arrived,” no arguments about interest.

WHAT WE BUILT THIS HACKATHON

Small enough to audit. *Real enough to trust.*

`program` Anchor — 7 instructions, 11 LiteSVM tests, deployed to devnet

`client` Codama-generated TypeScript client — type-safe end to end

`app` Next.js — Wallet Standard, cluster switch, full lifecycle UI, live overdue-fee meter

`demo` one command seeds mint, wallets, invoices, LP advance: `npm run demo:devnet`

`verified build` 9a44d237… — artifact ≡ devnet bytecode, re-checked in CI on every push

ROADMAP

From devnet demo to *payment rail.*

`KSeF import`	auto-create invoices from official e-invoice XML (2026 mandate)
`collection crank`	a bot collects on time, every time — zero-touch
`reputation passport`	on-chain paid / revoked / collected history per payer
`Token-2022`	multi-delegate and per-invoice mandate accounts
`recurring`	subscription-style invoices for retainer contracts

Try it in *two commands.*

repo	github.com/zuhdev/termin
program	GhEAovtiaJj48DwEwXWfDEqkxrPhbLfbryTHNifQEhZ1 (devnet)
video	[video URL]
run	npm install && npm run demo:devnet && npm run dev

Team: [names]

termin — the invoice that collects itself.